Doubly Linked List

1.Insertion at Beginning.

```c
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void insert()
{
    struct node *temp = head;

    while(temp != NULL)
    {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

void insertatbegin(int value)
{
    struct node *newnode;
```

```
Before Insertion:
301 <-> 201 <-> NULL
After Insertion at Beginning:
101 <-> 301 <-> 201 <-> NULL


=== Code Execution Successful =
```

```c
    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = value;
    newnode->prev = NULL;
    newnode->next = head;

    if(head != NULL)
    {
        head->prev = newnode;
    }

    head = newnode;
}

int main()
{
    insertatbegin(201);
    insertatbegin(301);

    printf("Before Insertion:\n");
    insert();

    insertatbegin(101);

    printf("After Insertion at Beginning:\n");
    insert();
    return 0;
}
```

```
Before Insertion:
301 <-> 201 <-> NULL
After Insertion at Beginning:
101 <-> 301 <-> 201 <-> NULL


=== Code Execution Successful
```

2.Insertion at End.

```c
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void insert()
{
    struct node *temp = head;

    while(temp != NULL)
    {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

void insertatend(int value)
{
    struct node *newnode, *temp;
```

```
Before Insertion:
101 <-> 201 <-> NULL
After Insertion at End:
101 <-> 201 <-> 301 <-> NULL


=== Code Execution Successful
```

```c
    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = value;
    newnode->next = NULL;

    if(head == NULL)
    {
        newnode->prev = NULL;
        head = newnode;
    }
    else
    {
        temp = head;
        while(temp->next != NULL)
        {
            temp = temp->next;
        }

        temp->next = newnode;
        newnode->prev = temp;
    }
}

int main()
{
    insertatend(101);
    insertatend(201);
```

```
Before Insertion:
101 <-> 201 <-> NULL
After Insertion at End:
101 <-> 201 <-> 301 <-> NULL


=== Code Execution Successful
```

```c
        temp->next = newnode;
        newnode->prev = temp;
    }
}

int main()
{
    insertatend(101);
    insertatend(201);

    printf("Before Insertion:\n");
    insert();

    insertatend(301);

    printf("After Insertion at End:\n");
    insert();
    return 0;
}
```

```
Before Insertion:
101 <-> 201 <-> NULL
After Insertion at End:
101 <-> 201 <-> 301 <-> NULL


=== Code Execution Successful
```

3. Insertion at Position.

```c
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void insert()
{
    struct node *temp = head;

    while(temp != NULL)
    {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

void insertatposition(int value, int pos)
{
    struct node *newnode, *temp;
```

```
Before Insertion:
101 <-> 201 <-> 401 <-> NULL
After Insertion at Position 3:
101 <-> 201 <-> 301 <-> 401 <-> NULL


=== Code Execution Successful ===
```

```c
    int i;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = value;

    if(pos == 1)
    {
        newnode->prev = NULL;
        newnode->next = head;
        if(head != NULL)
            head->prev = newnode;
        head = newnode;
        return;
    }

    temp = head;
    for(i = 1; i < pos-1; i++)
    {
        temp = temp->next;
    }

    newnode->next = temp->next;
    newnode->prev = temp;

    if(temp->next != NULL)
        temp->next->prev = newnode;

```

```
Before Insertion:
101 <-> 201 <-> 401 <-> NULL
After Insertion at Position 3:
101 <-> 201 <-> 301 <-> 401 <-> NULL


=== Code Execution Successful ===
```

```
54
55      temp->next = newnode;
56  }
57
58  int main()
59  {
60      insertatposition(101,1);
61      insertatposition(201,2);
62      insertatposition(401,3);
63
64      printf("Before Insertion:\n");
65     insert();
66
67      insertatposition(301,3);
68
69      printf("After Insertion at Position 3:\n");
70      insert();
71
72      return 0;
73  }
```

```
Before Insertion:
101 <-> 201 <-> 401 <-> NULL
After Insertion at Position 3:
101 <-> 201 <-> 301 <-> 401 <-> NULL


=== Code Execution Successful ===
```

4.Deletion at Beginning.

```c
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void delete()
{
    struct node *temp = head;

    while(temp != NULL)
    {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

void deleteatbegin()
{
    struct node *temp;
```

```
Before Deletion:
101 <-> 201 <-> 301 <-> NULL
After Deletion at Beginning:
201 <-> 301 <-> NULL


=== Code Execution Successful
```

```c
    if(head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    temp = head;
    head = head->next;

    if(head != NULL)
        head->prev = NULL;

    free(temp);
}

int main()
{
    struct node *n1,*n2,*n3;

    n1 = (struct node*)malloc(sizeof(struct node));
    n2 = (struct node*)malloc(sizeof(struct node));
    n3 = (struct node*)malloc(sizeof(struct node));

    n1->data = 101; n1->prev = NULL; n1->next = n2;
    n2->data = 201; n2->prev = n1; n2->next = n3;
    n3->data = 301; n3->prev = n2; n3->next = NULL;
```

```
Before Deletion:
101 <-> 201 <-> 301 <-> NULL
After Deletion at Beginning:
201 <-> 301 <-> NULL


=== Code Execution Successful
```

```
    n1 = (struct node*)malloc(sizeof(struct node));
    n2 = (struct node*)malloc(sizeof(struct node));
    n3 = (struct node*)malloc(sizeof(struct node));

    n1->data = 101; n1->prev = NULL; n1->next = n2;
    n2->data = 201; n2->prev = n1; n2->next = n3;
    n3->data = 301; n3->prev = n2; n3->next = NULL;

    head = n1;

    printf("Before Deletion:\n");
    delete();

    deleteatbegin();

    printf("After Deletion at Beginning:\n");
    delete();

    return 0;
}
```

```
Before Deletion:
101 <-> 201 <-> 301 <-> NULL
After Deletion at Beginning:
201 <-> 301 <-> NULL


=== Code Execution Successfu
```

5. Deletion at End.

```c
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void delete()
{
    struct node *temp = head;

    while(temp != NULL)
    {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

void deleteatend()
{
    struct node *temp;
```

```
Before Deletion:
101 <-> 201 <-> 301 <-> NULL
After Deletion at End:
101 <-> 201 <-> NULL


=== Code Execution Successfu
```

```c
    if(head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    temp = head;

    if(temp->next == NULL)
    {
        free(temp);
        head = NULL;
        return;
    }
    while(temp->next != NULL)
    {
        temp = temp->next;
    }

    temp->prev->next = NULL;
    free(temp);
}

int main()
{
    struct node *n1,*n2,*n3;

    n1 = (struct node*)malloc(sizeof(struct node));
```

```
Before Deletion:
101 <-> 201 <-> 301 <-> NULL
After Deletion at End:
101 <-> 201 <-> NULL


=== Code Execution Successful
```

```
    n1 = (struct node*)malloc(sizeof(struct node));
    n2 = (struct node*)malloc(sizeof(struct node));
    n3 = (struct node*)malloc(sizeof(struct node));

    n1->data = 101; n1->prev = NULL; n1->next = n2;
    n2->data = 201; n2->prev = n1; n2->next = n3;
    n3->data = 301; n3->prev = n2; n3->next = NULL;

    head = n1;

    printf("Before Deletion:\n");
    delete();

    deleteatend();

    printf("After Deletion at End:\n");
    delete();

    return 0;
}
```

```
Before Deletion:
101 <-> 201 <-> 301 <-> NULL
After Deletion at End:
101 <-> 201 <-> NULL


=== Code Execution Successful
```

6. Deletion at Position.

```c
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void delete()
{
    struct node *temp = head;

    while(temp != NULL)
    {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

void deleteatposition(int pos)
{
    struct node *temp;
```

```
Before Deletion:
101 <-> 201 <-> 301 <-> 401 <-> NULL
After Deletion at Position 3:
101 <-> 201 <-> 401 <-> NULL


=== Code Execution Successful ===
```

```c
    int i;

    if(head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    temp = head;

    if(pos == 1)
    {
        head = temp->next;
        if(head != NULL)
            head->prev = NULL;
        free(temp);
        return;
    }

    for(i = 1; i < pos; i++)
    {
        temp = temp->next;
    }

    if(temp->next != NULL)
        temp->next->prev = temp->prev;

```

```
Before Deletion:
101 <-> 201 <-> 301 <-> 401 <-> NULL
After Deletion at Position 3:
101 <-> 201 <-> 401 <-> NULL


=== Code Execution Successful ===
```

```
        temp->prev->next = temp->next;

    free(temp);
}

int main()
{
    struct node *n1,*n2,*n3,*n4;

    n1 = (struct node*)malloc(sizeof(struct node));
    n2 = (struct node*)malloc(sizeof(struct node));
    n3 = (struct node*)malloc(sizeof(struct node));
    n4 = (struct node*)malloc(sizeof(struct node));

    n1->data = 101; n1->prev = NULL; n1->next = n2;
    n2->data = 201; n2->prev = n1; n2->next = n3;
    n3->data = 301; n3->prev = n2; n3->next = n4;
    n4->data = 401; n4->prev = n3; n4->next = NULL;

    head = n1;

    printf("Before Deletion:\n");
    delete();

    deleteatposition(3);

    printf("After Deletion at Position 3:\n");
    delete();

    return 0;
}
```

```
Before Deletion:
101 <-> 201 <-> 301 <-> 401 <-> NULL
After Deletion at Position 3:
101 <-> 201 <-> 401 <-> NULL


=== Code Execution Successful ===
```